Thesis Notebook

2D Orbit with the suns density profile and mass function, along with dissipative forces

Setting functions up and choosing variables

In[∘]:=
```
(*R values in Schwarzschild coordinates*)
RStar = 8;(*Radius of the Star*)
M = 1;
```

Exterior Schwarzschild Isotropic coordinates definitions $r(\bar{r}), \bar{r}(r), A(\bar{r}), \alpha(\bar{r})$

TOV Isotropic coords definitions

Mass and Density functions from 2405.08113

Fixing the rho0 by letting the total mass integral equal to 1

In[∘]:=
```
rho[r_] := rho0 (1 - r/R)^6;
(*massIntegral =  4 Pi Integrate[rho[r] r^2, {r, 0, R}]; Set equal to 1*)
```

In[∘]:=
```
rhoCentre[R_] := 63/(Pi R^3) (*From the above*)
rho0 = rhoCentre[RStar];
```

In[∘]:=
```
MassFunc[r_] := Piecewise[{{252
      (r^9/(9 RStar^9) - 3 r^8/(4 RStar^8) + 15 r^7/(7 RStar^7) - 10 r^6/(3 RStar^6) + 3 r^5/RStar^5 - 3 r^4/(2 RStar^4) + r^3/(3 RStar^3)), r < RStar},
    {M, r ≥ RStar}
   }];
(*In 2405.08113 this function is divided by the solar mass, here I use M=1*)
```

New density function

In[∘]:=
```
rho[r_] := Piecewise[{
    {rho0 (1 - r/RStar)^6, r < RStar},
    {0, r ≥ RStar}
   }];
```

Getting new equations for the new interior metric:

In[∘]:=
```
(*TOV differential equation*)
dPdr[r_, Pressure_] := -(rho[r] + Pressure) ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

In[∘]:=
```
(*Phi differential equation for metric*)
dPhidrMetric[r_, Pressure_] := ((MassFunc[r] + 4 Pi Pressure r^3)/(r (r - 2 MassFunc[r])));
```

Numerically Integrate these two partially coupled differential equations

In[∘]:=
```
rmin = 10^-16 (*For these integrations*);
```

In[∘]:=
```
eq1 = P'[r] == dPdr[r, P[r]];
eq2 = Phi'[r] == dPhidrMetric[r, P[r]];
BCs = {P[RStar] == 0, Phi[RStar] == 1/2 Log[1 - 2 M/RStar]} ;
(*Phi[RStar] from the Schwarzschild metric*)
```

In[∘]:=
```
sol1 = NDSolve[
    {eq1,
     eq2,
     BCs},
    {P, Phi},
    {r, RStar, rmin}
   ][[1]]; (*Very small rmin to avoid singularity sarp mentioned*)
```

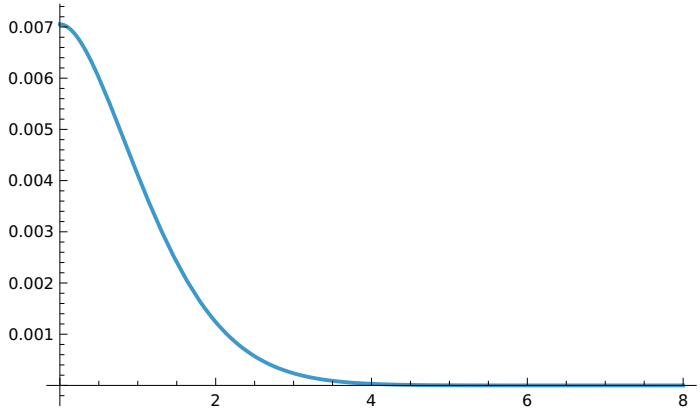
In[∘]:=
```
Pressure[r_] := P[r] /. sol1

PhiMetric[r_] = Phi[r] /. sol1;

(*Plot[Derivative[1][PhiMetric][r], {r, 0, RStar}]*)
(*Causing the issue it seems, but we have this Analytically!*)
```

In[∘]:=
```
Plot[Pressure[r], {r, 0, RStar}]
```

Out[∘]=



Solving for the new $\bar{r}$(r) and $r(\bar{r})$ inside, obtaining $r(\bar{r})$ via the interpolation trick Sarp explained

In[∘]:=
```
(*Defining the ODE*)
```

$$\text{drbardr}[\text{rbar_}, \text{r_}] := \frac{\text{rbar}}{\text{r}} \left(1 - \frac{2\,\text{MassFunc}[\text{r}]}{\text{r}}\right)^{-\frac{1}{2}}$$

Numerically Integrating the above equation.

In[∘]:=
```
eq = rbarofr'[r] == drbardr[rbarofr[r], r];
BC = {rbarofr[RStar] == rbarStar};
```

In[∘]:=
```
sol2 = NDSolve[
    {eq,
     BC},
    {rbarofr},
    {r, RStar, rmin}
   ][[1]];
```

In[∘]:=
```
rbarofrInside[r_] = rbarofr[r] /. sol2;
```

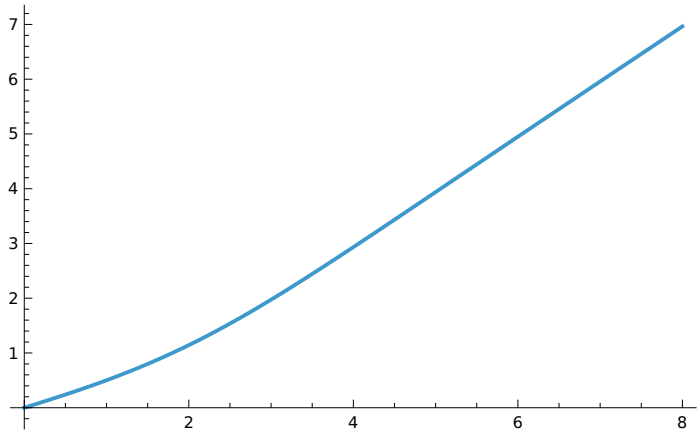
In[∘]:=
```
rbarofrInside[RStar]
rbarofrOutside[RStar] // N
```

Out[∘]= 6.9641

Out[∘]= 6.9641

In[∘]:=
```
Plot[rbarofrInside[r], {r, 0, RStar}]
```

Out[∘]=



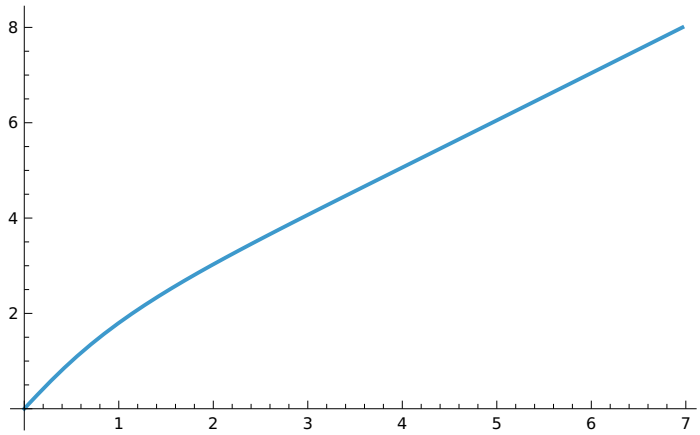
Now using the interpolation trick to obtain $r(\bar{r})$:

In[∘]:=
```
(*Generating the numerical data points:*)
(*Table[ expression, {variable, start, end, step}*)
data = Prepend[Table[
    {rbarofrInside[r], r},
    (*Here Mathematica stores rbar[r] and r as a 2 element list - first entry is
      the independent variable and the one after is the dependent variable*)
    {r, rmin, RStar, (RStar - rmin)/100 000}(*100000 points*)
   ], {0, 0}];
```

In[∘]:=
```
(*Evaluates ok at zero as sarp mentioned but integral doesn't like it one bit*)
```

In[∘]:=
```
(*Inverse Interpolation*)
rofrbarInside = Interpolation[data];
(*Builds a function from the data passing through all points unlike Fit which
 approximately matches the data to a function I give it with parameters,
FindFit finds the parameters for me given I guess a suitable function*)
```

In[∘]:=
```
Plot[rofrbarInside[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=


In[∘]:=
```
rofrbarInside[0]
```

Out[∘]= 0.

Writing clean full $r(\bar{r})$ equation to try and rid discontinuities

In[∘]:=
```
rofrbarFull[rbar_?NumericQ] := Piecewise[
   {
    {rofrbarInside[rbar], rbar < rbarStar},
    {rofrbarOutside[rbar], rbar ≥ rbarStar}
   }
  ];
```

Putting this together to find the new interior metric functions

finding alpha in terms of normal r first

In[∘]:=
```
dPhidrMetric[r_] := (MassFunc[r] + 4 Pi Pressure[r] r^3)/(r (r - 2 MassFunc[r]))
```

In[∘]:=
```
alpha[r_] := Exp[PhiMetric[r]]
DERalphaIns[r_] := dPhidrMetric[r] × alpha[r]
```

Exterior alpha derivative

In[∘]:=
```
DERαOutsiderbardummy = Derivative[1][α][rbar];
```

$$\text{DER}\alpha\text{Outsiderbar}[r_] := \left(\frac{1-\frac{1}{2\,\text{rbar}}}{2\left(1+\frac{1}{2\,\text{rbar}}\right)^2\text{rbar}^2}+\frac{1}{2\left(1+\frac{1}{2\,\text{rbar}}\right)\text{rbar}^2}\right) /.\ \text{rbar} \to \text{rbarofrOutside}[r]$$

```
(*More accurate subbing in after for rbar maybe?*)
```

$$\text{drbardr}[r_] := \frac{\text{rbarofrOutside}[r]}{r}\left(1-\frac{2\,\text{MassFunc}[r]}{r}\right)^{-\frac{1}{2}}$$

In[∘]:=
```
DERalpha[r_] := drbardr[r] × DERαOutsiderbar[r]
```

In[∘]:=
```
DERalpha[RStar] // N
DERalphaIns[RStar] // N
```

Out[∘]= 0.0180422

Out[∘]= 0.0180422

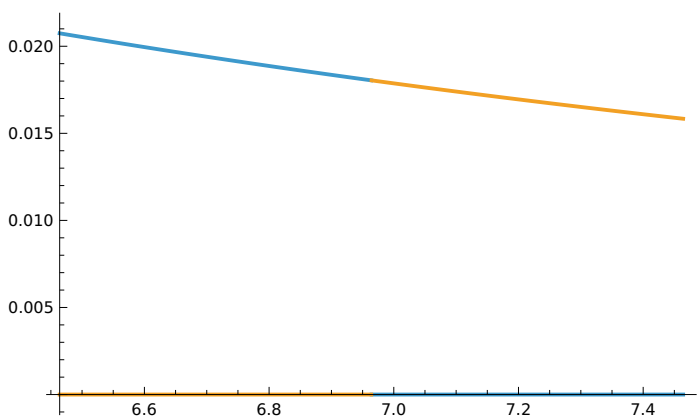
In[∘]:=
```
DERalpha[rbarofrOutside[RStar]] // N
DERalphaIns[rbarofrInside[RStar]] // N (*More or less continuous now but not exact*)
```

Out[∘]= 0.024422

Out[∘]= 0.0244215

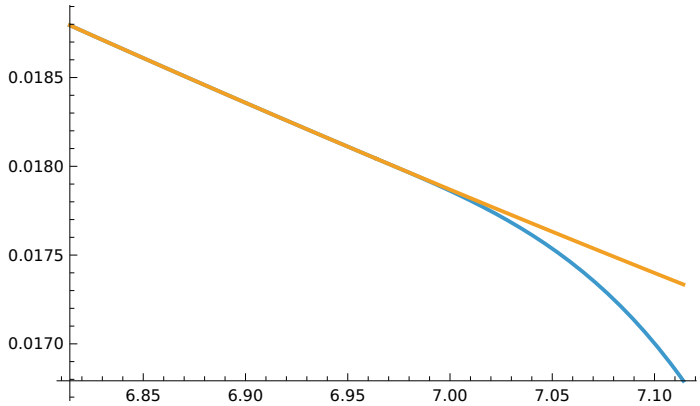
In[∘]:=
```
Plot[{Piecewise[{{DERalphaIns[rofrbarInside[rbar]], rbar ≤ rbarStar}}],
  Piecewise[{{DERalpha[rofrbarOutside[rbar]], rbar ≥ rbarStar}}]},
 {rbar, rbarStar - 0.5, rbarStar + 0.5}]
```

Out[∘]=



In[∘]:=
```
Plot[{DERalphaIns[rofrbarInside[rbar]], DERalpha[rofrbarOutside[rbar]]},
 {rbar, rbarStar - 0.15, rbarStar + 0.15}]
```

Out[∘]=



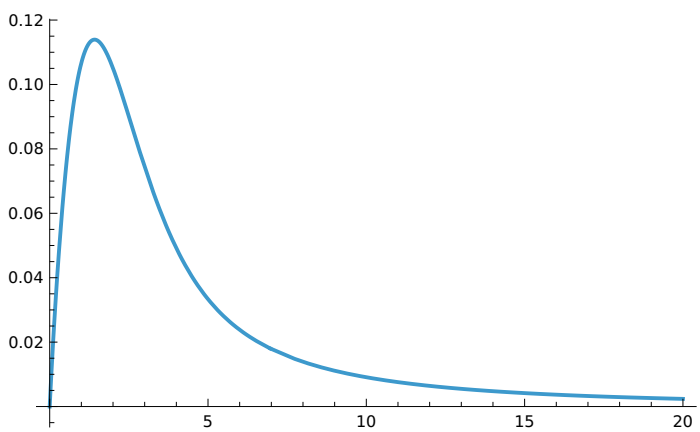
In[∘]:=
```
DERαOutside[rbar_] := drbardr[rofrbarOutside[rbar]] × DERαOutsiderbar[rofrbarOutside[rbar]]
DERαInside[rbar_] := dPhidrMetric[rofrbarInside[rbar]] × alpha[rofrbarInside[rbar]]
```

In[∘]:=
```
DERαFull[rbar_] := Piecewise[
  {
   {DERαInside[rbar], rbar < rbarStar},
   {DERαOutside[rbar], rbar ≥ rbarStar}
  }
 ]
```

In[∘]:=
```
(*DERαFull[rbar_]:=If[rbar<rbarStar,DERαInside[rbar],DERαOutside[rbar]]*)
```

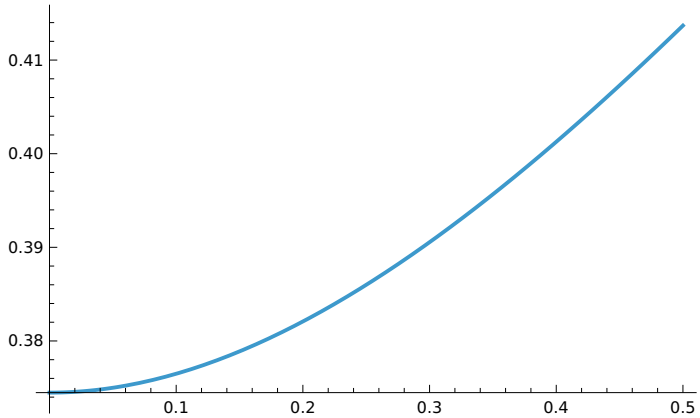
In[∘]:=
```
Plot[DERαFull[rbar], {rbar, 0, 20}]
```

Out[∘]=



In[∘]:=
```
αInside[rbar_?NumericQ] := Exp[PhiMetric[rofrbarInside[rbar]]]
```

In[∘]:=
```
Plot[αInside[rbar], {rbar, 0, .5}]
```

Out[∘]=


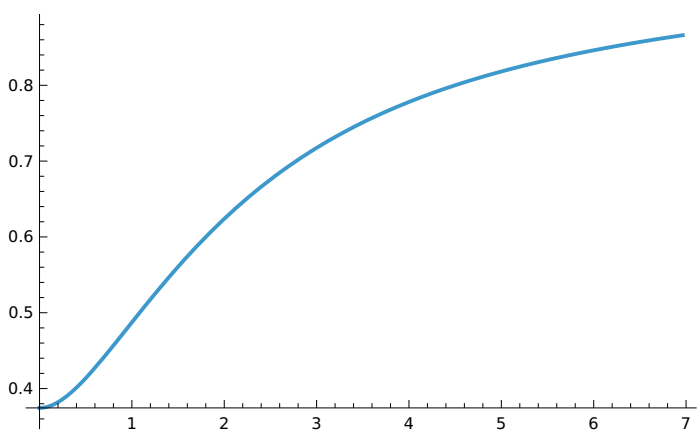
In[∘]:=
```
αInside[rbarStar]
α[rbarStar]
```

Out[∘]= 0.866025

Out[∘]= 0.866025

In[∘]:=
```
αFull[rbar_] := Piecewise[
  {
   {αInside[rbar], rbar < rbarStar},
   {α[rbar], rbar ≥ rbarStar}
  }
 ]
(*αFull[rbar_]:=If[rbar<rbarStar,αInside[rbar],α[rbar]]*)
```

In[∘]:=
```
Plot[αFull[rbar], {rbar, 0, rbarStar}]
```

Out[∘]=


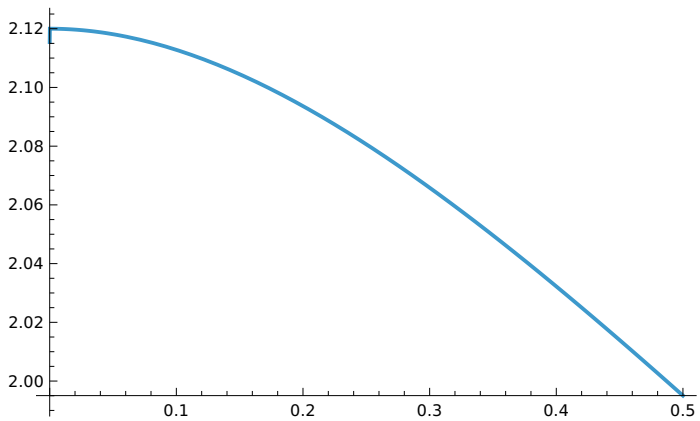
In[∘]:=
```
(*Plot[Derivative[1][αFull][rbar], {rbar, 0.4, rbarStar}]*) (*Phi metric causing the
 issue as it's the numerically integrating thing we are differentiating*)
```

Going to do the same derivative setup as the α component for the A component

In[∘]:=
```
AInside[rbar_?NumericQ] := rofrbarInside[rbar]/rbar;
```

In[∘]:=
```
Plot[AInside[rbar], {rbar, 0, .5}]
```

Out[∘]=



In[∘]:=
```
A[rbarStar]
AInside[rbarStar]
```

Out[∘]= 1.14875

Out[∘]= 1.14875

In[∘]:=
```
AFull[rbar_] := Piecewise[
   {
    {A[rbar], rbar ≥ rbarStar},
    {AInside[rbar], rbar < rbarStar}
   }
  ];
(*AFull[rbar_]:=If[rbar<rbarStar,AInside[rbar],A[rbar]]*)
```

In[∘]:=
```
drbardr[rbar_, r_] := rbar/r (1 - 2 MassFunc[r]/r)^(-1/2)
drdrbar[rbar_] := rofrbarInside[rbar]/rbar (1 - 2 MassFunc[rofrbarInside[rbar]]/rofrbarInside[rbar])^(1/2)
```

In[∘]:=
```
DERAInside[rbar_] := drdrbar[rbar]/rbar - rofrbarInside[rbar]/rbar^2;
DERAOutside[rbar_] := Derivative[1][A][rbar]
```

In[∘]:=
```
DERAFull[rbar_] := Piecewise[
   {
    {DERAOutside[rbar], rbar ≥ rbarStar},
    {DERAInside[rbar], rbar < rbarStar}
   }
  ];
(*DERAFull[rbar_]:=If[rbar<rbarStar,DERAInside[rbar],DERAOutside[rbar]]*)
```

In[∘]:=
```
DERAOutside[rbarStar]
DERAInside[rbarStar]
```

Out[∘]= -0.0220995

Out[∘]= -0.0220995

Density and Pressure in isotropic coordinates

In[∘]:=
```
rho0 = rhoCentre[RStar];
```

In[∘]:=
```
rho[rbar_] := Piecewise[
   {
    {rho0 (1 - rofrbarInside[rbar]/RStar)^6, rbar < rbarStar},
    {0, rbar ≥ rbarStar}
   }
  ];
```

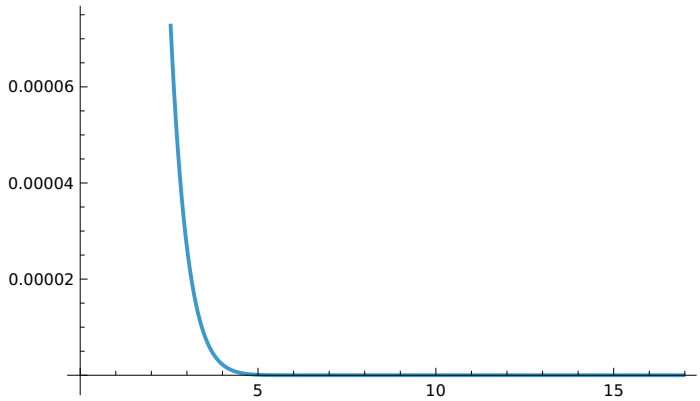
In[∘]:=
```
(*Maybe issue with NDSolve is that there are two numerically
 integrated things here being used and its causing issues?*)
(*Going to make the above as one interpolating function*)
data = Table[{rbarofrInside[r], Pressure[r]},
   {r, 0, RStar, RStar/1000}];
```

In[∘]:=
```
IsotropicCoordsPressure = Interpolation[data];
```

In[∘]:=
```
(*Solved the issue of the solver not being able
 to get an equation for rbar'[t] (method residual error)*)
```

In[∘]:=
```
Plot[IsotropicCoordsPressure[rbar], {rbar, 0, rbarStar + 10}]
```

Out[∘]=



Sorting out the speed of sound as a function of r: a(r)

In[∘]:=
```
(*Getting the derivative of the density*)
rhodummy[r_] := rho0 (1 - r/RStar)^6
rhodummy'[r];
drhodr[r_] := -6 rho0/RStar (1 - r/RStar)^5
```

In[∘]:=
```
cutoff = .999 RStar
```

Out[∘]= 7.992

Speed of sound in isotropic coordinates

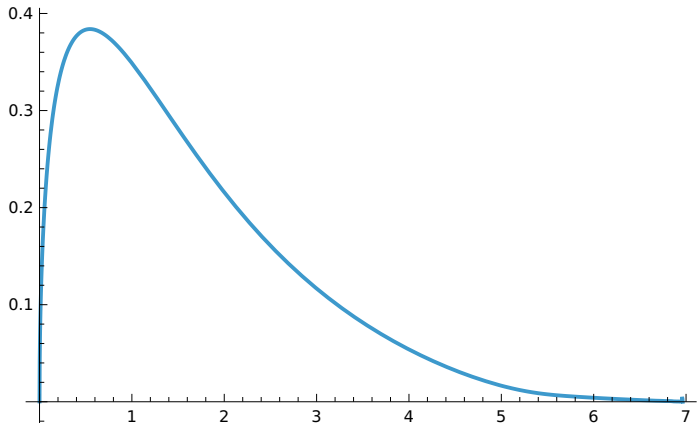
In[∘]:=
```
cutoffIso = rbarofrInside[cutoff]
```

Out[∘]= 6.95606

```
(*Needed to implement these measures to speed things up using
 Module to avoid recomputation and also the N@ to return numbers
 instead of symbols for NDSolve, pressure was causing an issue*)
(*Module avoids repeating the calculation three times as
 there were three arguments I would have had to have inputted it in*)
```

In[∘]:=
```
asosPiecewise[rbar_?NumericQ] := Module[{r = rofrbarInside[rbar]}, Piecewise[{
    {N@Sqrt[dPdr[r, Pressure[r]] / drhodr[r]], rbar < cutoffIso},
    {N@Sqrt[(M (RStar - r)) / (6 RStar (RStar - 2 M))], cutoffIso ≤ rbar < rbarStar},
    {10^-6, rbar ≥ rbarStar}
   }
  ]
 ]
```

In[∘]:=
```
Plot[asosPiecewise[rbar], {rbar, 0, rbarStar}]
asosPiecewise[rbarStar]
```

Out[∘]=


Out[∘]= $\frac{1}{1\,000\,000}$

Setting up 2D ODEs for new density profile:

Constants that are going to be used

In[∘]:=
```
m = 10^-6; (*mass of PBH?*)
λGR[rbar_] := 1.49 * asosPiecewise[rbar]
q = 2;
```

Mach number interpolation

Interpolating

In[∘]:=
```
fMachLow[m_] := 1/2 Log[(1 + m)/(1 - m)] - m ;
fMachhigh[m_] := 1/2 Log[1 - 1/m^2] + 10
```

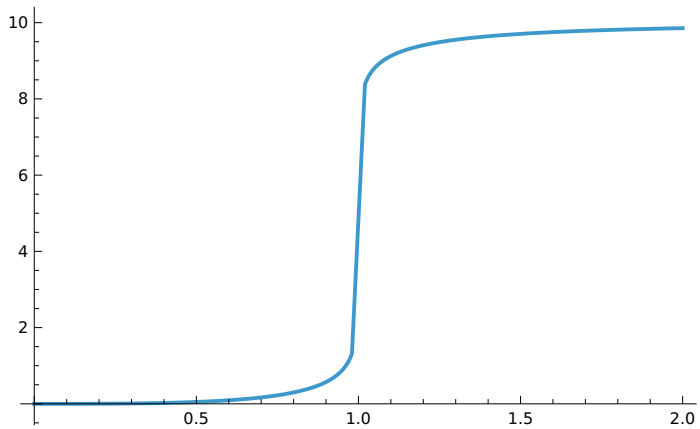
In[∘]:=
```
x1 = .98;
x2 = 1.02;
y1 = fMachLow[x1];
y2 = fMachhigh[x2];
```

In[∘]:=
```
anaylticline[Mach_] := y1 + (y2 - y1)(Mach - x1)/(x2 - x1)
```

In[∘]:=
```
IVariableInterpolated[Mach_] := Piecewise[{
    {1/2 Log[(1 + Mach)/(1 - Mach)] - Mach, Mach < 0.98},
    {anaylticline[Mach], 0.98 < Mach < 1.02},
    {1/2 Log[1 - 1/Mach^2] + 10, Mach > 1.02}
  }];
```

In[∘]:=
```
Plot[IVariableInterpolated[Mach], {Mach, 0, 2}, Exclusions → None, PlotRange → All]
```

Out[∘]=



Outside Star

Velocities

In[∘]:=
$$\text{Ut}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{1 + \left(\frac{\text{ur}}{\text{A[rbar]}}\right)^2 + \left(\frac{\text{uphi}}{\text{rbar A[rbar]}}\right)^2}}{\alpha[\text{rbar}]};$$

$$\text{umagOutside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{\sqrt{\left(\text{ur}^2 + \left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{A[rbar]}}$$

Rates of change of velocities

In[∘]:=
$$\text{drBardt}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{1}{\text{A[rbar]}^2\, \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\, \text{ur};$$

$$\text{dphidtOutside}[\text{rbar_}, \text{ur_}, \text{uphi_}] := \frac{1}{\text{rbar}^2 * \text{A[rbar]}^2 * \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\, \text{uphi};$$

Gravitational Waves

```
In[•]:= FRRr[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(15 r^4)*drBardt[rbar, ur, uphi]*
     (8 M^2 + r M (6 drBardt[rbar, ur, uphi]^2 + 36 r^2 dphidtOutside[rbar, ur, uphi]^2)))];

In[•]:= FRRphi[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarOutside[rbar]}, ((8 m^2)/(5 r^4)*dphidtOutside[rbar, ur, uphi]*
     (2 M^2 + r M (9 drBardt[rbar, ur, uphi]^2 - 6 r^2 dphidtOutside[rbar, ur, uphi]^2)))];
```

In[•]:= duphidtOutside[m_, rbar_, ur_, uphi_] := $\frac{\eta}{m}$ (A[rbar]^2 rbar^2 FRRphi[m, rbar, ur, uphi]);

durdt2DOutside[m_, rbar_, ur_, uphi_] :=

$$-\alpha[\text{rbar}] \times \text{Ut}[\text{rbar}, \text{ur}, \text{uphi}] \times \text{DER}\alpha\text{Outside}[\text{rbar}] + \frac{\text{ur}^2}{\text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\left(\frac{\text{DERAOutside}[\text{rbar}]}{\text{A}[\text{rbar}]^3}\right) + \frac{\text{uphi}^2}{\text{Ut}[\text{rbar}, \text{ur}, \text{uphi}]}\left(\frac{1}{\text{rbar}^3 * \text{A}[\text{rbar}]^2} + \frac{\text{DERAOutside}[\text{rbar}]}{\text{rbar}^2 * \text{A}[\text{rbar}]^3}\right) + \frac{\eta}{m}\left(\text{A}[\text{rbar}]^2\ \text{FRRr}[m, \text{rbar}, \text{ur}, \text{uphi}]\right);$$

In[•]:= (*$\frac{\eta}{m}$(A[rbar]^2 rbar^2 FRRphi[rbar, ur, uphi])
+ $\frac{\eta}{m}$(A[rbar]^2 FRRr[rbar, ur, uphi])*)

Inside Star

Velocities

In[•]:= UtInside[rbar_, ur_, uphi_] := $\frac{\sqrt{1 + \left(\frac{\text{ur}}{\text{AInside}[\text{rbar}]}\right)^2 + \left(\frac{\text{uphi}}{\text{rbar AInside}[\text{rbar}]}\right)^2}}{\alpha\text{Inside}[\text{rbar}]}$;

umagInside[rbar_, ur_, uphi_] := $\frac{\sqrt{\left(\text{ur}^2 + \left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{AInside}[\text{rbar}]}$

Mach business

In[•]:= Mach[rbar_, ur_, uphi_] := $\frac{\frac{\sqrt{\left(\text{ur}^2+\left(\frac{\text{uphi}}{\text{rbar}}\right)^2\right)}}{\text{AInside}[\text{rbar}]}}{\text{asosPiecewise}[\text{rbar}] \times \alpha\text{Inside}[\text{rbar}] \times \text{UtInside}[\text{rbar}, \text{ur}, \text{uphi}]}$

Dissipative Forces

Accretion

In[∘]:=
```
(*q=2, Γ=2*)
mdot[m_, rbar_, ur_, uphi_] :=
  1.49 * (4 Pi rho0 m^2) / (asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2);
```

In[∘]:=
```
dpdtaccretionr[m_, rbar_, ur_, uphi_] :=
  -((4 Pi αInside[rbar] (1.49) (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1 / (asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2)) ur;

dpdtaccretionphi[m_, rbar_, ur_, uphi_] :=
  -((4 Pi αInside[rbar] (1.49) (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2))
   (1 / (asosPiecewise[rbar]^2 + (umagInside[rbar, ur, uphi] / (α Inside[rbar] × UtInside[rbar, ur, uphi]))^2))
   uphi (*a's simplify in this regime*)
```

Drag

In[∘]:=
```
dpdtdragphi[m_, rbar_, ur_, uphi_] := 1 / umagInside[rbar, ur, uphi]^3 (-4 π αInside[rbar] *
    (rho[rbar] + IsotropicCoordsPressure[rbar]) * m^2 * (1 + 2 umagInside[rbar, ur, uphi]^2)^2 *
    IVariableInterpolated[Mach[rbar, ur, uphi]] * uphi);

dpdtdragr[m_, rbar_, ur_, uphi_] :=
  (-4 π αInside[rbar] (rho[rbar] + IsotropicCoordsPressure[rbar]) m^2
    (1 + 2 umagInside[rbar, ur, uphi]^2)^2 IVariableInterpolated[Mach[rbar, ur, uphi]] *
    ur) / umagInside[rbar, ur, uphi]^3;
```

Rates of change of velocities

In[∘]:=
```
drBardtInside[rbar_, ur_, uphi_] := 1/(AInside[rbar]^2 UtInside[rbar, ur, uphi]) ur;

dphidtInside[rbar_, ur_, uphi_] := 1/(rbar^2 AInside[rbar]^2 UtInside[rbar, ur, uphi]) uphi;
```

In[∘]:=
```
(*Redefining the mass function without the piecewise component for simplicity*)
MassFunction[r_] :=
  252 (r^9/(9 RStar^9) - 3 r^8/(4 RStar^8) + 15 r^7/(7 RStar^7) - 10 r^6/(3 RStar^6) + 3 r^5/RStar^5 - 3 r^4/(2 RStar^4) + r^3/(3 RStar^3));
```

In[∘]:=
```
Mp[r_] := Derivative[1][MassFunction][r]
Mpp[r_] := Derivative[2][MassFunction][r]
Mppp[r_] := Derivative[3][MassFunction][r]
```

In[∘]:=
```
FRRrIns[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarInside[rbar]}, (8 m^2)/(15 r^4) * drBardtInside[rbar, ur, uphi] *
    (8 MassFunction[r]^2 + r MassFunction[r] (6 drBardtInside[rbar, ur, uphi]^2 +
          36 r^2 dphidtInside[rbar, ur, uphi]^2 + Mp[r] - 3 r Mpp[r]) -
      r^2 (Mp[r]^2 + 3 r^2 dphidtInside[rbar, ur, uphi]^2 (6 Mp[r] - r Mpp[r]) +
          drBardtInside[rbar, ur, uphi]^2 (2 Mp[r] + r Mpp[r] - r^2 Mppp[r])))];
```

In[∘]:=
```
FRRphiIns[m_, rbar_, ur_, uphi_] :=
  Module[{r = rofrbarInside[rbar]}, (8 m^2)/(5 r^4) * dphidtInside[rbar, ur, uphi] *
    (2 MassFunction[r]^2 + 2 r^4 dphidtInside[rbar, ur, uphi]^2 × Mp[r] + r MassFunction[r]
       (9 drBardtInside[rbar, ur, uphi]^2 - 6 r^2 dphidtInside[rbar, ur, uphi]^2 - 2 Mp[r]) -
      r^2 drBardtInside[rbar, ur, uphi]^2 (7 Mp[r] - 2 r Mpp[r]))];
```

In[∘]:=
```
duphidtInside[m_, rbar_, ur_, uphi_] :=
  η/m (dpdtaccretionphi[m, rbar, ur, uphi] + dpdtdragphi[m, rbar, ur, uphi] +
     AInside[rbar]^2 rbar^2 FRRphiIns[m, rbar, ur, uphi]);

durdt2DInside[m_, rbar_, ur_, uphi_] :=
  -αInside[rbar] × UtInside[rbar, ur, uphi] × DERαInside[rbar] +
   ur^2/UtInside[rbar, ur, uphi] (DERAInside[rbar]/AInside[rbar]^3) +
   uphi^2/UtInside[rbar, ur, uphi] (1/(rbar^3 AInside[rbar]^2) + DERAInside[rbar]/(rbar^2 AInside[rbar]^3)) +
   η/m (dpdtaccretionr[m, rbar, ur, uphi] + dpdtdragr[m, rbar, ur, uphi] +
     AInside[rbar]^2 FRRrIns[m, rbar, ur, uphi]);
```

In[∘]:=
```
(* + 0*AInside[rbar]^2 FRRrIns[rbar, ur, uphi]
  + 0*AInside[rbar]^2 rbar^2 FRRphiIns[rbar, ur, uphi]*)
```

Defining Energy and Angular momentum per unit mass functions along with other orbital parameters

In[∘]:=
```
EnergyPerUnitMass[ptilde_, eccen_] := Sqrt[((ptilde - 2)^2 - 4 * eccen^2)/(ptilde * (ptilde - 3 - eccen^2))];

LPerUnitMass[ptilde_, M_, eccen_] := (ptilde * M)/Sqrt[ptilde - 3 - eccen^2];

ptildefunc[p_, Mass_] := p/Mass;

rapoapsis[eccen_, ptilde_, Mass_] := ((ptilde) * Mass)/(1 - eccen);

(*In Schwarzschild areal r coordinates*)
rperiapsis[eccen_, ptilde_] := ptilde/(1 + eccen);
```

Defining the constants to define the motion

Eccentricity and semi-latus rectum

In[∘]:=
```
eccentricity = 0.4;
(*eccentricity = 9/11; (*Obtained from setting periapsis to 5.5 and ptilde still to 10*)*)
semilatrecP = 10;
ptilde = ptildefunc[semilatrecP, M];
```

Energy and Angular momentum per unit mass

In[∘]:=
```
Energy = EnergyPerUnitMass[ptilde, eccentricity];
AngularMomentumUphi0 = LPerUnitMass[ptilde, M, eccentricity];
```

Initial drop radius (apoapsis), closest point it goes to (perihelion), defining Isotropic star radius

```
r0 = rapoapsis[eccentricity, ptilde, M];
rbar0 = rbarofrOutside[r0];

rperiapsis[eccentricity, ptilde];
(*Setting it up such that it passes the point where I know the accretion is
 strong enough to sufficiently slow it down from the inside simulation.
  (Needed this for accretion and drag to work independently and together, GW allowed
     me to go back to it not having to pass through the star on the first orbit)
 Used this to find the proper rapoapsis*)
```

In[∘]:=
```
(*r_a + r_p = 2a, a = p/(1-e^2) = 30.25
    r_a = 60.5-5.5 = 55 = rbar0 = r_apoapsis*)
```

Must satisfy this inequality to pass through star

In[∘]:=
```
ptilde < RStar (1 + eccentricity)
ptilde > 6 + 2 eccentricity
```

Out[∘]= True

Out[∘]= True

Solving the ODEs

In[∘]:=
```
drBardtInside[rbarStar, 0, AngularMomentumUphi0]
durdt2DInside[m, rbarStar, 0, AngularMomentumUphi0]
duphidtInside[m, rbarStar, 0, AngularMomentumUphi0]
dphidtInside[rbarStar, 0, AngularMomentumUphi0]
```

Out[∘]= 0.

Out[∘]= 0.00219943

Out[∘]= $-5.4786 \times 10^{-9}\ \eta$

Out[∘]= 0.0466817

In[∘]:=
```
t0 = 0;
tEnd = 50 000;
η = 7.25 × 10^3;

solIn = NDSolve[
    {
     rbar'[t] ==
      Piecewise[
       {
        {drBardt[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {drBardtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     ur'[t] ==
      Piecewise[
       {
        {durdt2DOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {durdt2DInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     uphi'[t] ==
      Piecewise[
       {
        {duphidtOutside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {duphidtInside[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
       }
      ],
     phi'[t] ==
      Piecewise[
       {
```

```
        {dphidtOutside[rbar[t], ur[t], uphi[t]], rbar[t] ≥ rbarStar},
        {dphidtInside[rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
      }
    ],
  mass'[t] ==
   Piecewise[
    {
      {0, rbar[t] ≥ rbarStar},
      {mdot[mass[t], rbar[t], ur[t], uphi[t]], rbar[t] < rbarStar}
    }
   ],
  rbar[t0] == rbar0,
  ur[t0] == 0,
  phi[t0] == 0,
  uphi[t0] == AngularMomentumUphi0,
  mass[t0] == m,

  WhenEvent[rbar[t] ≤ 10^-2 , "StopIntegration"]
 },
 {rbar, ur, phi, uphi, mass},
 {t, t0, tEnd},
 MaxSteps → Infinity,
 WorkingPrecision → MachinePrecision
 (*Machine precision and 20k max steps also works*)
]][[1]];
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

General: Further output of InterpolatingFunction::dmval will be suppressed during this calculation.

In[∘]:=
```
rbarNumIn = rbar /. solIn;
urNumIn = ur /. solIn;
phiNumIn = phi /. solIn;
uphiNumIn = uphi /. solIn;
massNumIn = mass /. solIn;
```

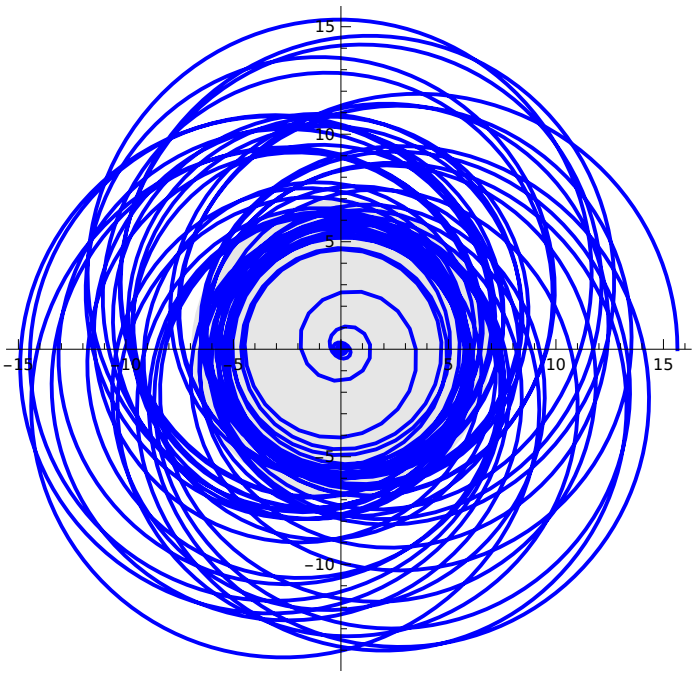
In[∘]:=
```
tFinish = rbarNumIn["Domain"][[1, 2]]
tSurface = t /. FindRoot[rbarNumIn[t] == rbarStar, {t, 10, 10, tFinish}]
```

Out[∘]= 9552.67

Out[∘]= 159.819

In[∘]:=
```
x[t_] := rbarNumIn[t] Cos[phiNumIn[t]];
y[t_] := rbarNumIn[t] Sin[phiNumIn[t]];
orbit2D = ParametricPlot[{x[t], y[t]}, {t, t0, 0.999 tFinish}, AspectRatio → Automatic,
   AxesLabel → {"x", "y"}, PlotRange → All, PlotStyle → {Thick, Blue}];
(*Automatic scale makes the aces the same scale*)
centralBody = Graphics[{GrayLevel[0.9], Disk[{0, 0}, rbarStar]}];
Show[centralBody, orbit2D, AspectRatio → Automatic, PlotRange → All, Axes → True]
```

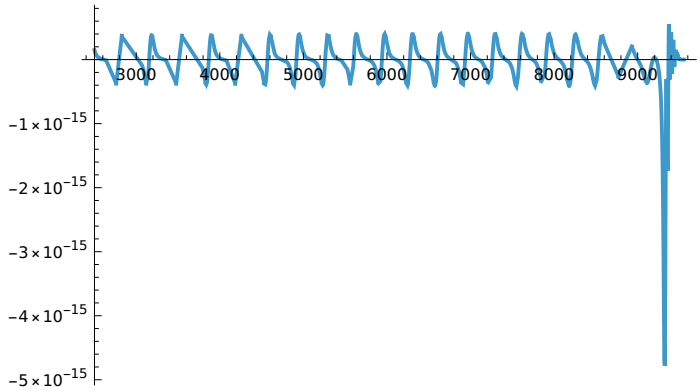
Out[∘]=



In[∘]:=
```
FRRrFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRr[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRrIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

In[∘]:=
```
Plot[FRRrFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

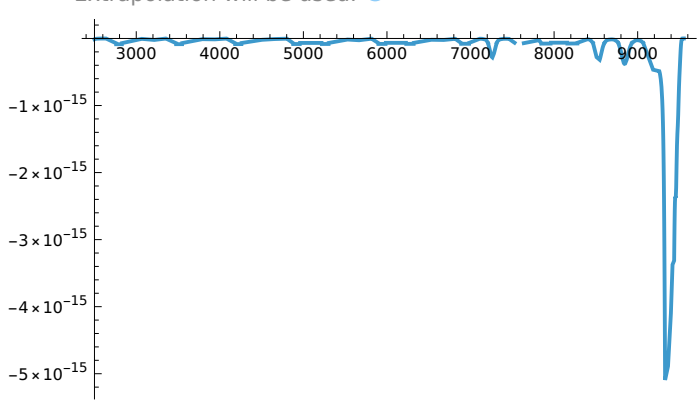
InterpolatingFunction: Input value {9.46828} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=


In[∘]:=
```
FRRphiFull[m_, rbar_, ur_, uphi_] := Piecewise[
  {
   {FRRphi[m, rbar, ur, uphi], rbar ≥ rbarStar},
   {FRRphiIns[m, rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

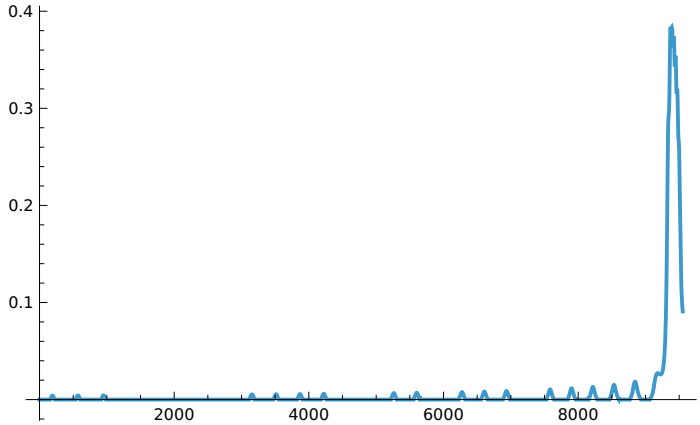
In[∘]:=
```
Plot[FRRphiFull[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
 {t, 2500, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {9.46828} lies outside the range of data in the interpolating function. Extrapolation will be used.

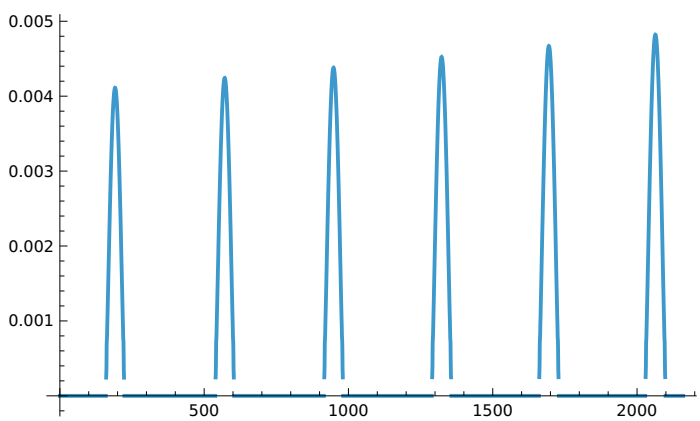
Out[∘]=


In[∘]:=
```
Plot[asosPiecewise[rbarNumIn[t]], {t, 0, tFinish}, PlotRange → All]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=


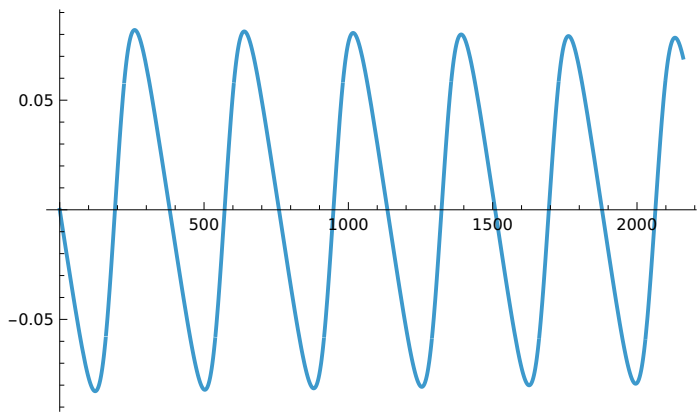
In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {0, rbarNumIn[t] ≥ rbarStar},
    {asosPiecewise[rbarNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

Out[∘]=


In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {drBardt[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {drBardtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

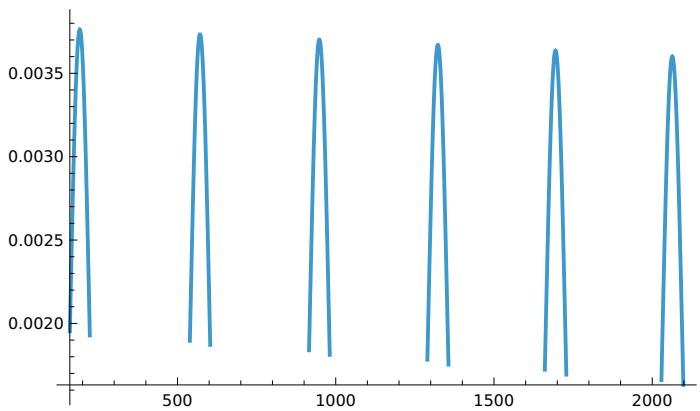
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {durdt2DOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {durdt2DInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

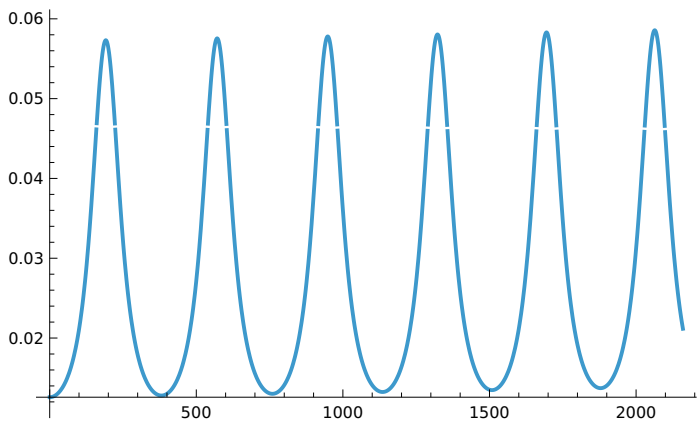
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {dphidtOutside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] ≥ rbarStar},
    {dphidtInside[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

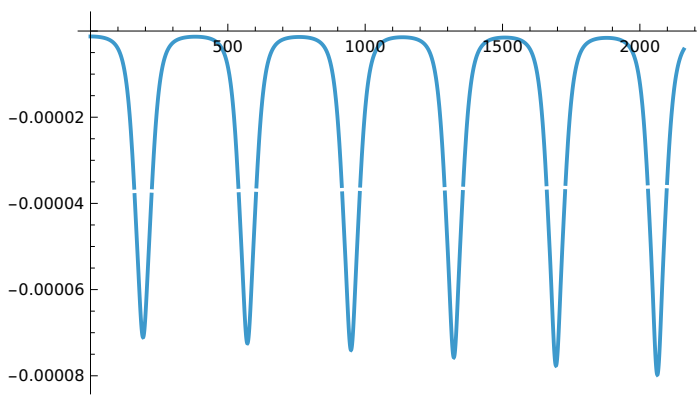
Out[∘]=



In[∘]:=
```
Plot[
 {
  Piecewise[
   {
    {duphidtOutside[massNumIn[t], rbarNumIn[t], urNumIn[t], uphiNumIn[t]],
     rbarNumIn[t] ≥ rbarStar},
    {duphidtInside[massNumIn[t],
       rbarNumIn[t], urNumIn[t], uphiNumIn[t]], rbarNumIn[t] < rbarStar}
   }
  ]
 },
 {t, 0, tSurface + 2000}, PlotRange → All
]
```

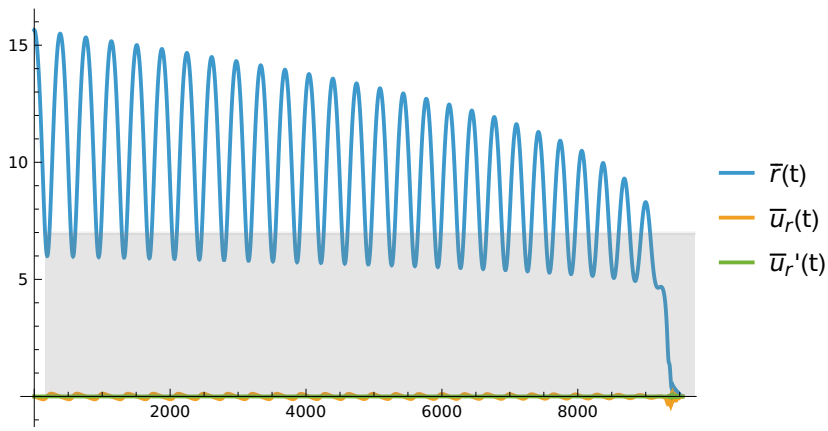
InterpolatingFunction: Input value {15.6507} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[∘]=



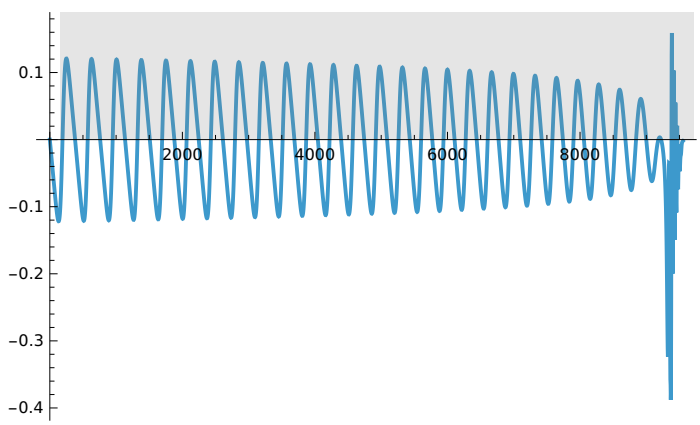
In[∘]:=
```
outFigRadial = Show[Plot[{rbarNumIn[t], urNumIn[t], urNumIn'[t]},
    {t, t0, .999 tFinish}, PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
   PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

Out[∘]=

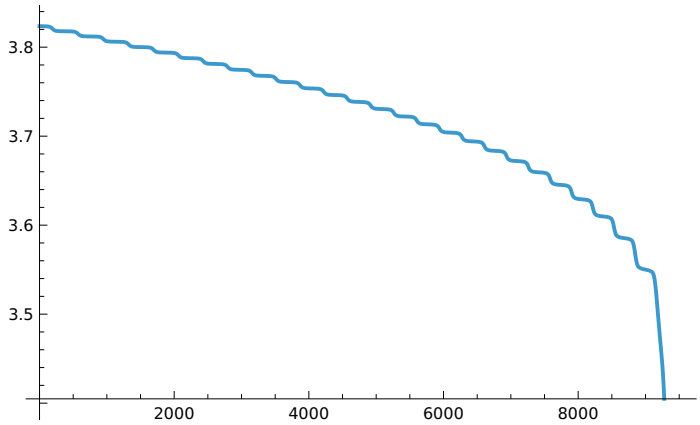


In[∘]:=
```
outFigRadial = Show[Plot[{urNumIn[t]}, {t, t0, tFinish},
    PlotLegends → LineLegend[Automatic, {"r̄(t)", "ū_r(t)", "ū_r'(t)"}]],
  Plot[rbarStar, {t, tSurface, tSurface + tFinish},
   PlotStyle → Directive[Gray, Opacity[0.1]], Filling → Bottom]]
```

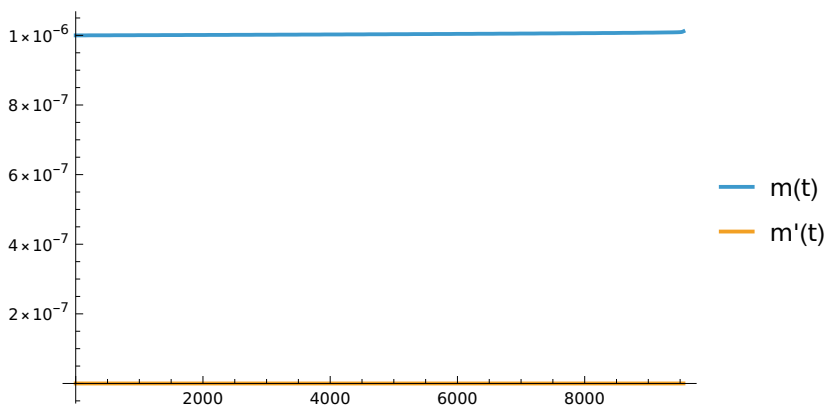
Out[∘]=



In[∘]:=
```
outFigAngular = Plot[{uphiNumIn[t]}, {t, t0, tFinish},
  PlotLegends → LineLegend[Automatic, {"phi(t)", "uphi(t)", "uphi'(t)"}]]
```

Out[∘]=


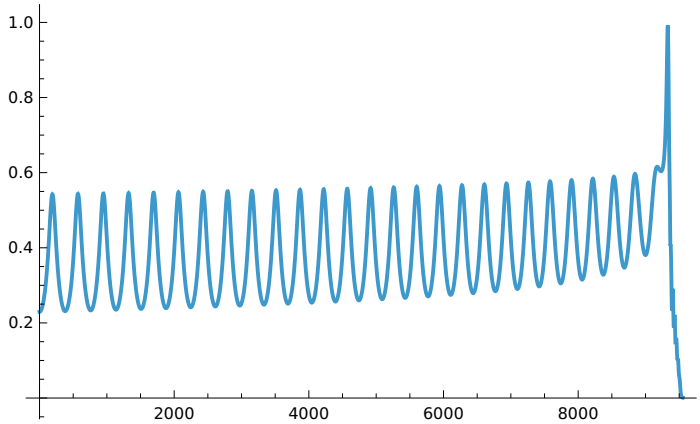
In[∘]:=
```
outFigMass = Show[Plot[{massNumIn[t], massNumIn'[t]},
   {t, t0, tFinish}, PlotLegends → LineLegend[Automatic, {"m(t)", "m'(t)"}]]]
```

Out[∘]=


In[∘]:=
```
umag[rbar_, ur_, uphi_] := Piecewise[
  {
   {umagOutside[rbar, ur, uphi], rbar ≥ rbarStar},
   {umagInside[rbar, ur, uphi], rbar < rbarStar}
  }
 ]
```

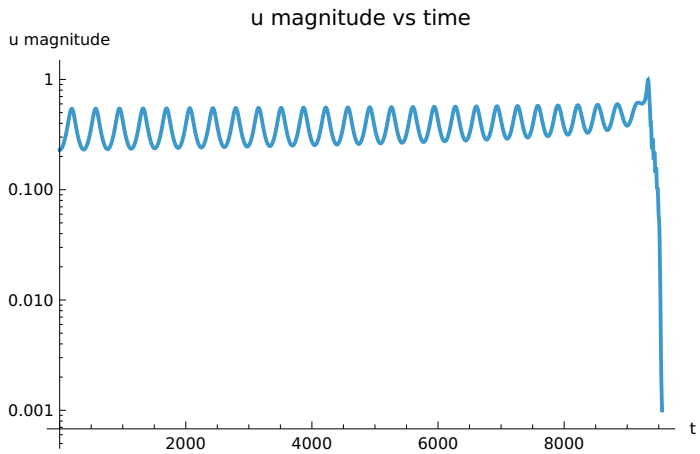
In[∘]:=
```
Plot[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]], {t, 0, tFinish}, Exclusions → None]
(*Normal behaviour judging from rho=const simulation*)
```

Out[∘]=



In[∘]:=
```
LogPlot[Evaluate[umag[rbarNumIn[t], urNumIn[t], uphiNumIn[t]] /. solIn], {t, 0, tFinish},
 PlotRange → All, AxesLabel → {"t", "u magnitude"}, PlotLabel → "u magnitude vs time"]
```

Out[∘]=



In[∘]:=
```
(*Show me the value of u at tfinish*)
umag[rbarNumIn[tFinish], urNumIn[tFinish], uphiNumIn[tFinish]] /. solIn
```

Out[∘]= 0.000995684